



Actividad 2

Estructuras de Datos

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/ACDiag_2
- **Fecha máxima de entrega:** Jueves 20 de agosto 17:10
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Objetivos de la actividad

- Implementar distintos tipos de estructuras *built-in* según corresponda.
- Resolver un problema real de manejo de inventario con estructuras de datos.
- Familiarizarse con el uso de herramientas del curso y la entrega de actividades en Github.

Introducción

El dueño de la tienda online DCCompras te ha pedido que lo ayudes creando un programa que maneje el inventario de productos y los pagos online.

El sistema debe poder manejar el precio y cantidad disponible de cada producto y también debe poder calcular eficientemente los valores a pagar cuando un cliente quiere pagar su carro.

En esta actividad se espera que implementes código que utilice las estructuras de datos apropiadas para manejar los datos de manera eficiente.

Input Productos

Se te dará un inventario inicial de productos en un archivo `productos.txt`. En cada línea del archivo se indicará el nombre, precio y cantidad del producto disponible separados por coma.

Por ejemplo:

```
Polera Gris L,15800,3
Polera Gris S,15800,1
Bufanda rallada,28300,2
```

Además se te dará un archivo `acciones.txt` que indican las acciones realizadas por los clientes online en el orden en el que suceden. Cada línea del archivo parte con el nombre de la acción, luego el RUT del usuario y en el caso de la acción "Agregar al carro" hay un tercer valor correspondiente el nombre del producto.

Input Acciones

A continuación se explican todas las acciones posibles de los clientes. Para cada acción tu programa deberá imprimir un mensaje dependiendo del resultado de la acción. En el mensaje debe estar incluido el RUT del usuario. Las posibles acciones son:

- **Agregar al carro:** El usuario con el RUT indicado agrega una unidad del producto indicado a su carro. Luego de esta acción, el producto queda reservado para el usuario hasta que este termine la compra o cierre su sesión.

Si no existe el producto o no quedan productos disponibles, se debe imprimir un mensaje apropiado.

Si se realiza la acción con éxito, se debe imprimir un mensaje con el nombre del producto.

- **Sacar del carro:** El usuario con el RUT indicado elimina el último producto que fue agregado al carro y se devuelve al inventario.

Si el carro no tiene productos, imprime un mensaje de apropiado.

Si se realiza la acción con éxito, se imprime el nombre del producto devuelto.

- **Cerrar sesión:** El usuario con el RUT indicado cierra su sesión y libera todos los productos reservados de su carro de vuelta al inventario. No se cobra nada al usuario y su carro se vacía. El usuario puede volver agregar productos a su carro después de esta acción.

Si el carro estaba vacío, debe imprimirse un mensaje de error.

Si se realiza la acción con éxito debe imprimirse el nombre de todos los productos liberados.

- **Pagar:** El usuario con el RUT indicado paga los productos de su carro. Todos los productos del carro pasan a estar fuera del inventario.

Si no hay productos en el carro, se debe imprimir un mensaje que indique el problema.

Si la acción se realiza con éxito debes imprimir el monto total de la compra.

Por ejemplo un posible archivo de acciones dado el inventario anterior puede ser:

```
Agregar al carro,20000000-0,Polera Gris S
Agregar al carro,21123123-5,Bufanda rallada
Agregar al carro,21123123-5,Polera Gris S
Sacar del carro,18453533-4
Cerrar sesion,20000000-0
Agregar al carro,21123123-5,Polera Gris S
Pagar,21123123-5
Pagar,18453533-4
```

Output

Eres libre de elegir el formato exacto del programa, pero debe cumplir con los requisitos explicados en la sección de acciones. Un ejemplo de output del programa dados los archivos de input en en la sección anterior sería:

```
[20000000-0] Polera Gris S agregado al carro
[21123123-5] Bufanda rallada agregado al carro
[21123123-5] No quedan unidades de Polera Gris S
[18453533-4] No quedan productos en el carro para sacar
[20000000-0] Sesión cerrada. Productos liberados ['Polera Gris S']
[21123123-5] Polera Gris S agregado al carro
[21123123-5] Pagado carro. Total de compra: 44100
[18453533-4] No hay nada que pagar
```

Ejecución de código

Se entrega un archivo `main.py` con lo mínimo para leer los archivos línea por línea para facilitar la lectura de los archivos. Tu código debe poder ejecutarse por consola como `python main.py` (O variando `python` por `python3`).

También se entregan 3 carpetas con archivos de productos y acciones. Se recomienda probar tu programa con el ejemplo chico y avanzar al mediano y grande.

Se espera que tu programa maneje eficientemente el inventario de la tienda y el contenido de los carros de los clientes usando las estructuras de datos aprendidas.

Notas

- No puedes hacer *import* de otras librerías externas a las entregadas en el archivo `main.py`
- Recuerda que la ubicación de tu entrega es en **tu repositorio de Git**. En la rama (*branch*) por defecto del repositorio: `main`.
- Recuerda que si bien esta actividad no tiene nota, si la subes a Github va a ser revisada por un ayudante y te entregará feedback al respecto.
- Siéntete libre de agregar nuevos `print` en cualquier lugar para revisar objetos, modificaciones, etc... Es una herramienta muy útil para encontrar errores. Y si aparece un error inesperado, ¡léelo! Intenta interpretarlo.

- Se recomienda que modifique los archivos de productos y acciones para probar tu código en distintas situaciones.